

Question Bank — Week 12

Heaps and Priority Queues (Lecture 12) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	2
Part B — Short answer and essay	5
Part C — Trace the code	5
Part D — Heap state — draw every step	6
Part E — Complexity analysis	6
Part F — Find and fix the bug	7
Part G — Write the code	8

Answers are in a separate file: [week12-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. MinHeap, MaxHeap and PriorityQueue are the classes of `dsa/heap.py`. A heap is stored in an array `_items`, indexed from 0: the children of index i are $2i + 1$ and $2i + 2$, its parent is $(i - 1) // 2$. `push` appends and sifts up; `pop` moves the last item to the root and sifts down, swapping with the **better** child; `heapify` sifts down from index $n // 2 - 1$ back to 0. `PriorityQueue.enqueue(item, priority)` pushes the triple `(priority, counter, item)`; a lower number is served first. Unless a question says otherwise, **Q** is the min-heap

`Q = [1, 5, 3, 8, 6, 4, 9, 11, 10]`

of 9 values, at indices 0 to 8.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W12-M01 – W12-M22
B	Short answer and essay	W12-E1 – W12-E5
C	Trace the code	W12-T1 – W12-T5
D	Heap state — draw every step	W12-S1 – W12-S3
E	Complexity analysis	W12-K1 – W12-K3
F	Find and fix the bug	W12-B1 – W12-B4
G	Write the code — checked by pytest	W12-C1 – W12-C5

Part A — Multiple choice

W12-M01 [what] In a **min**-heap, which statement is always true?

- **a)** the array is sorted in ascending order
- **b)** every parent is less than or equal to its children
- **c)** every left child is less than or equal to its sibling
- **d)** the largest value is at the last index

W12-M02 [what] With 0-based indices, the **left** child of index i is at:

- **a)** $2i$
- **b)** $i + 1$
- **c)** $2i + 1$
- **d)** $2i + 2$

W12-M03 [how] The parent of index 10 is at index:

- **a)** 4
- **b)** 5
- **c)** 3
- **d)** 9

W12-M04 [how] The height of a heap holding 1,000 values (a single node has height 0) is:

- **a)** 10
- **b)** 500
- **c)** 31
- **d)** 9

W12-M05 [how] Which array is a valid **min**-heap?

- **a)** [1, 3, 2, 7, 4]
- **b)** [1, 3, 2, 2, 4]
- **c)** [2, 1, 3]
- **d)** [1, 4, 2, 3]

W12-M06 [how] `Q.push(2)`: how many swaps does the sift-up make?

- **a)** 0
- **b)** 1
- **c)** 2
- **d)** 3

W12-M07 [how] `Q.pop()` returns 1. Which value is at the root afterwards?

- **a)** 5
- **b)** 3
- **c)** 10
- **d)** 4

W12-M08 [why] Sift-down swaps an item with the **better** of its two children, not just any child that beats it, because:

- **a)** it is faster to compare with the better child
- **b)** it keeps the tree complete
- **c)** the left child might not exist
- **d)** the child that moves up becomes the parent of its sibling, so it must beat the sibling too

W12-M09 [what] `heapify` on an array of n values sifts down first at index:

- **a)** 0
- **b)** $n - 1$
- **c)** $n // 2 - 1$
- **d)** $n // 2 + 1$

W12-M10 [what] The worst-case cost of `heapify` on n values is:

- **a)** $O(n)$
- **b)** $O(n \log n)$
- **c)** $O(\log n)$
- **d)** $O(n^2)$

W12-M11 [why] `heapify` costs less than n pushes because:

- **a)** it never looks at half of the values
- **b)** most nodes are near the bottom, and a sift-down costs only the levels **below** a node
- **c)** it uses sift-up, which is cheaper than sift-down
- **d)** the input is usually almost sorted

W12-M12 [how] Pushing the values $n, n - 1, \dots, 2, 1$ (in that order) into an empty **min**-heap costs, in total:

- **a)** $O(n)$
- **b)** $O(\log n)$
- **c)** $O(n^2)$
- **d)** $O(n \log n)$

W12-M13 [what] `peek()` on a heap of n values costs:

- **a)** $O(\log n)$
- **b)** $O(1)$
- **c)** $O(n)$
- **d)** $O(n \log n)$

W12-M14 [what] Checking whether an arbitrary value x is in a heap of n values costs, in the worst case:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n \log n)$

W12-M15 [why] A heap is stored in an array, with no node objects, because:

- **a)** Python lists are faster than objects
- **b)** a heap is always sorted
- **c)** arrays can grow and nodes cannot
- **d)** its shape is complete, so there are no gaps, and index arithmetic replaces the child and parent references

W12-M16 [how] `enqueue("a", 2)`, `enqueue("b", 1)`, `enqueue("c", 1)`, then three `dequeue()`s return:

- **a)** a, b, c
- **b)** b, c, a
- **c)** c, b, a
- **d)** b, a, c

W12-M17 [why] `PriorityQueue` pushes `(priority, counter, item)` rather than `(priority, item)` because:

- **a)** it makes `dequeue` faster
- **b)** it counts the items in the queue
- **c)** when two priorities tie, Python would compare the items, which may not be comparable
- **d)** it turns the min-heap into a max-heap

W12-M18 [how] The k largest of n values, found with a min-heap of at most k values, cost:

- **a)** $O(n \log k)$
- **b)** $O(k \log n)$
- **c)** $O(n \log n)$
- **d)** $O(nk)$

W12-M19 [why] To find the k **largest** values you keep a **min**-heap of size k because:

- **a)** a max-heap cannot hold k values
- **b)** a min-heap is faster than a max-heap
- **c)** the smallest values must be found first
- **d)** its root is the weakest of the best k so far — the one to throw out when a better value arrives

W12-M20 [what] The extra space of heap sort (Week 10) is:

- **a)** $O(n)$
- **b)** $O(\log n)$
- **c)** $O(1)$
- **d)** $O(n \log n)$

W12-M21 [why] Heap sort sorts into **ascending** order with a **max**-heap because:

- **a)** each extraction swaps the maximum to the end of the heap, which is where it belongs in the sorted array
- **b)** a max-heap is stable

- **c)** a min-heap cannot be built in place
- **d)** a max-heap needs fewer comparisons

W12-M22 [how] After `MaxHeap([3, 1, 4, 1, 5, 9, 2, 6])`, the value at `_items[1]` is:

- **a)** 5
 - **b)** 1
 - **c)** 4
 - **d)** 6
-

Part B — Short answer and essay

W12-E1 [why] (4 marks) Define a binary heap by its two properties. Explain why the **shape** property matters: what it gives the height, and what it gives the storage.

W12-E2 [how] (4 marks) Describe push and pop on a min-heap, including sift-up and sift-down, the cost of each, and the special cases the code must handle.

W12-E3 [why] (4 marks) Explain how `heapify` builds a heap bottom-up, why it must go from the last parent **backwards**, and show that it is $O(n)$. Why are n pushes $O(n \log n)$ in the worst case?

W12-E4 [why] (3 marks) Compare four ways to implement a priority queue — unsorted array, sorted array, binary heap, balanced binary search tree — by the cost of enqueue, dequeue and peek. Which would you choose, and why?

W12-E5 [why] (3 marks) `PriorityQueue` stores (`priority`, `counter`, `item`). What goes wrong with (`priority`, `item`)? What does the counter add besides avoiding that error?

Part C — Trace the code

W12-T1 [how] Starting from an empty `MinHeap`, push 6, 2, 8, 1, 5, 3, then pop twice. Give the array after every operation, and each value returned.

W12-T2 [how] Trace `MinHeap([3, 9, 2, 1, 4, 5, 8, 7, 6, 0])`: for every `sift_down(i)` that `heapify` calls, give the array afterwards. How many swaps are made in all?

W12-T3 [how] Pop from `Q` three times. Give each value returned and the array after each pop.

W12-T4 [how] Trace this `PriorityQueue`. Give the heap array (as triples) after each operation, and each value returned.

```
pq = PriorityQueue()
pq.enqueue("print report", 3)
```

```
pq.enqueue("fix login bug", 1)
pq.enqueue("reply to email", 2)
pq.dequeue()
pq.enqueue("restart server", 1)
pq.enqueue("update docs", 3)
pq.dequeue(); pq.dequeue(); pq.dequeue()
```

W12-T5 [how] Trace `heap_sort` (Week 10) on `[4, 10, 3, 5, 1]`: the array after the build phase, and after each swap-and-sift-down.

Part D — Heap state — draw every step

W12-S1 [how] Starting from an empty MaxHeap, push 5, 12, 7, 20, 3, 15. Draw the tree **and** the array after each push, and mark each swap.

W12-S2 [how] Find the 3 largest of `[5, 1, 9, 3, 7, 2, 8]` with a min-heap of at most 3 values, as in Lecture 12's top-k. For each value, say whether it is pushed, replaces the root, or is skipped, and draw the heap array afterwards.

W12-S3 [why] Draw each array as a tree and say whether it is a valid **min**-heap. For each that is not, mark every parent-child pair that breaks the property.

- (a) `[1, 2, 3, 4, 5, 6]`
 - (b) `[1, 3, 2, 5, 4, 1]`
 - (c) `[2, 2, 2, 3, 2]`
 - (d) `[4, 5, 6, 7, 8, 9, 3]`
-

Part E — Complexity analysis

W12-K1 [how] Give Θ and justify, for a list values of n numbers:

```
h = MinHeap()
for x in values:
    h.push(x)
result = [h.pop() for _ in range(len(values))]
```

What does `result` contain? How does it compare with heap sort in time and space?

W12-K2 [how] Give the cost of top-k (the k largest of n values) with a min-heap of size k , and compare it with sorting everything and taking k . Which is better for $k = 10$ and $n = 10^6$? For $k = n$?

W12-K3 [why] A student writes `heapify forwards`:

```
def heapify(self):
    for index in range(len(self._items) // 2):
        self._sift_down(index)
```

Is it correct? Give an input on which it fails, or prove that it works. What does a forward loop of `self._sift_up(index)` over **every** index cost, and is that one correct?

Part F — Find and fix the bug

W12-B1 [how]

```
def _sift_up(self, index):
    items = self._items
    while index > 0:
        parent = index // 2
        if not self._beats(items[index], items[parent]):
            return
        items[index], items[parent] = items[parent], items[index]
        index = parent
```

W12-B2 [why]

```
def _sift_down(self, index):
    items = self._items
    size = len(items)
    while True:
        left, right = 2 * index + 1, 2 * index + 2
        if left < size and self._beats(items[left], items[index]):
            child = left
        elif right < size and self._beats(items[right], items[index]):
            child = right
        else:
            return
        items[index], items[child] = items[child], items[index]
        index = child
```

W12-B3 [how]

```
def pop(self):
    if len(self._items) == 0:
        raise IndexError("pop from an empty heap")
    last = self._items.pop()
    best = self._items[0]
    self._items[0] = last
    self._sift_down(0)
```

```
return best
```

W12-B4 [how]

```
def heapify(self):  
    for index in range(len(self._items) // 2 - 1, 0, -1):  
        self._sift_down(index)
```

Part G — Write the code

In `practice/week12.py`; check with `pytest tests/test_practice_week12.py -v`. Use your own `MinHeap` and `MaxHeap` as the working storage — not `sorted`, `list.sort` or `heapq`. Some tests count your comparisons.

W12-C1 [how] `is_min_heap(values)` — is a plain list a valid min-heap? $O(n)$.

W12-C2 [why] `top_k(values, k)` — the k largest, largest first, in $O(n \log k)$. Why a **min**-heap?

W12-C3 [how] `merge_sorted(lists)` — merge k sorted lists in $O(N \log k)$. What does each heap entry hold?

W12-C4 [why] `join_ropes(lengths)` — the cheapest total cost of joining ropes, two at a time, where a join costs the sum of the two lengths. Why is “always join the two shortest” right?

W12-C5 [why] `running_medians(values)` — the median after each value, with two heaps. Why do the two roots give the median?